

What's going on in DB, and why

Victor W. Marek

Department of Computer Science
University of Kentucky

Spring 2014

- ▶ Databases as the result of existence of *stable storage*
 - ▶ Tape
 - ▶ Disk
 - ▶ Floppy Disk
 - ▶ CD R-W
 - ▶ Jump Drive
 - ▶ Flash drive
 - ▶ ...

- ▶ Database theory and practice are based on abstract treatment of stable storage where implementation details are hidden from the user
- ▶ Database Management System is such abstraction
- ▶ Prehistory: Navigational Databases (network databases and hierarchical databases based on various data structures such as double-linked lists etc.)
- ▶ Relational DBMS (Codd, 1970) - embedding database into *logic*
- ▶ System R implemented 1974/75 at IBM
- ▶ RDBMSes - dominant database paradigm for over 25 years

- ▶ We all have a *built-in* understanding of *logic* in its simplest form
- ▶ Relational concepts with its reference to a simple data structure (a rectangular table with the first row containing metadata) are very intuitive
- ▶ It is possible to explain to *people in the office* basic relational operations
- ▶ The declarative description of queries (“Here is what I want”) can be easily compiled into a procedural description (“Here is how I want it to be done”) via so-called Relational Algebra
- ▶ This results in a *query plan*

- ▶ This last step can be *optimized*
- ▶ This resulted in very fast operation *once the data is in the main memory*
- ▶ (What is the issue?)
- ▶ Also, techniques for storage of DB information were improved ($B+$ trees, what is it?)
- ▶ Indexing (actually developed before RDBMSes) speeded up retrieval
- ▶ (But multiple indexes has disadvantages, too. Why?)

- ▶ The theory of *transactions* for RDBMSes was developed (transactions are crucial for financial applications, but for many other applications, too, for instance medical applications)
- ▶ ACID properties of transactions will be discussed in this course (Atomicity, Consistency, Isolation, and Durability)

- ▶ The theory of *crash recovery* was developed (absolutely necessary in financial applications, but again needed in many other areas.)
- ▶ Various forms of *security* were added to RDBMSes
- ▶ (The basic security model of RDBMS is *discretionary access control*)

- ▶ But “pure” relational RDBMS turned out to be impractical
- ▶ Generally, *Relational Model* treats descriptors (values of attributes) atomically (i.e. they have no further structure)
- ▶ But this is *obviously* not true; for instance strings have structure
- ▶ We can not forfeit *regular expressions* in processing
- ▶ Other important data types have structure (for instance *time*)

- ▶ The simplicity of relational query language (there is now just one, Structured Query Language, out of IBM, and owned by US National Institute for Science and Technology) quickly disappeared
- ▶ Competing vendors of RDBMSes (IBM, Oracle, Microsoft, Sybase, and several other vendors that since disappeared) extended the SQL standard in various directions
- ▶ While few vendors dominate the market right now (Oracle, IBM, SAP) there are *many* relational database management systems, both proprietary and open-source

- ▶ SQL includes all three basic database languages:
 - ▶ Data definition language (“what is to be stored”)
 - ▶ Data manipulation language (“how is the data changed when updates are made, “how we insert data”, “how we delete data”)
 - ▶ Query language (“How do I ask for what I want”)
- ▶ DDL deals with *metadata* that is data about *schemes*
- ▶ DML deals with data manipulation
- ▶ QL describes queries

- ▶ But this was just the beginning - plenty of other aspects of DB work were added (for instance: aggregate queries, dealing with metadata (for instance scheme transformation), security, setting up indexes and transactions, active component, specialized libraries (“blades”), recursive queries and other aspects of database work)

Relational databases, cont'd

[More data](#)[Big data](#)

- ▶ SQL standard is now enormous
- ▶ Current version (the eight one) is called SQL-2011
- ▶ No RDBMS supports entire SQL, and specific implementations differ widely
- ▶ Besides of commercial RDBMSes there are community-supported implementations, for instance very popular MySQL, (but it is a complex story, and next year it may no longer be publicly available, as its current owner, Oracle Inc. promised to keep it publicly available only to 2015)
- ▶ Still, there are plenty of open-source and free alternatives to MySQL
- ▶ (MySQL was a part of LAMP suite - Linux, Apache, MySQL and Perl (or Python, or PHP))

- ▶ The relational tables enforce some version of *first normal form*
- ▶ (That is attributes mostly have no structure)
- ▶ The data types of SQL are quite limited, and originally user-defined types were not accepted
- ▶ But there is often need to have complex attributes (think about record with the value of *Iname:marek* having multiple addresses)
- ▶ In other words records (chunks of information stored in DB) may be complex objects
- ▶ Various hacks were proposed (such as *mapping on relational tables*)

RDBM as a “straitjacket”, cont’d

- ▶ Also, what will we do if the data is really big (think Google, Yahoo!, etc., but also see below)
- ▶ And what will we do if we just want to process streaming data (such as data on the University communications)?
- ▶ And what if data has a complex structure (say we store complex documents divided in chapters, sections, paragraphs, etc.)
- ▶ Object-relational databases were proposed and implemented, for instance *Postgres* Object-relational DBMS out of UC Berkeley and still available on Linux

Data grows and so is storage

More data

Big data

- ▶ Originally, both processing and storage were expensive
- ▶ The first sign of change was gradual decrease in the cost of storage
- ▶ When less expensive (but also less reliable) disks appeared database community reacted with the *RAID standards*
- ▶ Important RAID levels are RAID 1 and RAID 5
- ▶ Roughly speaking: data is “mirrored” (duplicated on two or more disks) and striped (i.e. partitioned on different disks on byte level, possibly with checksums (what is it?))
- ▶ Also error-correcting codes (i.e. multiple checksums) used

- ▶ Throughout 1980ies and part of 1990ies storage was still a problem and the prevailing idea of RDBMSes was to *decompose* tables (this process was called *normalization*)
- ▶ The idea is that one stores less, computes more (reconstructing original tables via joins)
- ▶ Important class of normalized databases: *Third normal form* databases

- ▶ Soon storage ceased to be a serious problem
- ▶ Also the computing power (via so-called “Moore Law”) increased and the cost of processors started to go down
- ▶ Finally, the laws of physics intervened and faster (single-core) processors no longer possible
- ▶ New generations of processors (multi-core) appeared
- ▶ (And we no longer normalize, either)
- ▶ Database appliances are now available

- ▶ Data often comes from several sources
- ▶ Think police: there is a database of fugitives, databases of stolen cars, database of individuals with multiple convictions etc.
- ▶ But this data may be needed *together* - think cruising police notice a car with the plates that is in the database of stolen cars
- ▶ They may want to see if the driver is a fugitive (maybe it is fugitive *marek*)
- ▶ This situation, when data from different sources may be presented together is usually called *federated database*
- ▶ The software handling federated database is called *mediator*

- ▶ Normally, databases are *passive*, i.e. the user initiates database processing
- ▶ But modern databases often possess *active component*
- ▶ *Events* (such as specific transactions) that satisfy *conditions* trigger various *actions*
- ▶ For instance user *marek* tries to take more than \$1000.00 from his account - we roll back this transaction
- ▶ Another example (also from banking industry) overdraft handling
- ▶ In medical applications: *marek* is late for his annual check-up

But the world changed in the meanwhile

- ▶ The Web changed many things (*How did you even function when there were no Web?*)
- ▶ Data became global
- ▶ There was, suddenly, need for processing much bigger data
- ▶ Huge number of databases became accessible from far away
- ▶ e-commerce emerged
- ▶ There was need for searching *very large* and poorly structured repositories of information
- ▶ These searches could not really be done by means of RDBMSes
- ▶ The issue is *not* the speed of the search (in fact once the data is in RDBMS the processing is fast) but the sheer volume of data (and its variability)

- ▶ Data is globalized, often internationalized (various languages, for instance)
- ▶ Documents (i.e. data items) may have very different structure
- ▶ (It is easy to represent relational data in HTML, but the other direction is not easy)
- ▶ Even when data is uniform it can be huge (think data that comes from geophysical satellites, meteorological data, network operations data)
- ▶ Relatively small operations (such as our University IT) generate very large data - so big that it is not possible to store it (for instance network traffic data)

- ▶ *Big Data* is a term for dealing with large collections of data
- ▶ Often not structured (i.e. not easily represented in tabular form)
- ▶ More often than not stored in big *data centers* with thousands of processing machines and disks
- ▶ These machines likely “lower end”, likewise with disks
- ▶ So MTTF (what is it?) lower and so data needs to be duplicated
- ▶ Often difficult to process with traditional tools
- ▶ But containing valuable information relevant to the business process of companies

- ▶ Credit Card transaction information
- ▶ Client information in large companies
- ▶ Product sales information
- ▶ Genetic information
- ▶ Environmental information
- ▶ Stock-market information

Example: *marek* goes to Kroger

- ▶ Let us look at a common scenario
- ▶ An individual (we call him *marek*) goes to a supermarket to buy groceries
- ▶ It is not the first one he is there
- ▶ *marek* finds what he needs and now he is at checkout
- ▶ *marek* takes out his credit card and pays
- ▶ What happens?

marek goes to Kroger, cont'd

- ▶ The cash register at checkout is networked and connects to Kroger data center
- ▶ *marek* credit card provides data on *marek*
- ▶ But also, enticed by savings, *marek* brings out his Kroger's store card
- ▶ Now the data center computes various information on *marek*, for instance the fact that *marek* often buys chocolate, actually *Ghirardelli* brand
- ▶ *marek* gets his discounts (from his store card) but also he gets *coupons* for future savings on chocolate

marek goes to Kroger, cont'd

- ▶ The idea is that Kroger, by learning *marek's* habits can compute an information that, likely, will entice *marek* to visit again!
- ▶ Without having networked checkouts and storing information on *marek* in their data center, Kroger would not be able to do this
- ▶ The key points in our scenario were the ability of tracing *marek* (either via his credit card(s), or via his store card, or both)
- ▶ This scenario, with various variations became a routine

- ▶ Data sometimes comes in *streams*
- ▶ Example: network data
- ▶ (Defense against DoS and DDoS attacks (what is it?) requires stream processing)
- ▶ Example: Geophysical data
- ▶ Example stock market data
- ▶ There is a version of SQL for streams (look up ESPER (<http://www.espertech.com>))

- ▶ Large merchants such as *ebay* or *Amazon* routinely use big data to present opportunities to visitors
- ▶ Minor vendors often buy potential visitor's information
- ▶ (I made an experiment: I searched on Google for *oak wood flooring*; afterwards for months I would get ads on oak wood flooring)
- ▶ All this means that Google must have sold the fact that my computer was involved in such search to advertisement agencies
- ▶ Thus that there is no *free lunch* - *marek* did not pay for search of the Web; somebody else (ultimately companies selling hardwood flooring) paid for it

But what supports such capabilities?

- ▶ Of course databases
- ▶ In case of Kroger it was directly tying *marek* to his favorite purchases, but also maybe more; the database was *mined* and if it was established that chocolate purchasers often like to buy broccoli (there may be such *statistical correlation*) then *marek* would get a coupon for future broccoli purchase
- ▶ The examples we gave indicate that there is more to Big Data; statistics is prominently present there

- ▶ The class of software handling *Big Data* is called *Analytics*
- ▶ Example: Big cereal company watches sales of its products to see where they need to spend their advertisement money (why do they do that?)
- ▶ Clearly database (IT) people do not *design* analytics, but often have to implement it
- ▶ Analytics reside somewhere on the border of IT and business, but now form a significant part of IT effort, and use significant part of database support

What did we learn?

More data

Big data

- ▶ Databases have an old history (if anything in CS can have such history)
- ▶ They changed quite a lot in the process
- ▶ Databases underlie many (if not most) applications
- ▶ There are various classes of databases (relational, object-relational, noSQL)
- ▶ When data is becoming *really big* standard database paradigms (such as relational) are not enough